

A

 $A \Rightarrow B$ $\therefore B$

(modus ponens)

To prove an algorithm is correct, we must show:

① That it terminates.

How? usually, runtime analysis Θ notation

sometimes, use a decrementing fn

(we'll come back to this)

② Partial Correctness:

If it terminates, then it is correct.

How? Loop / recursion invariant

↑
today

$\therefore$ If we show ① and ②,
then we know the algo. is correct.

PEASANT MULTIPLY (x, y)

```

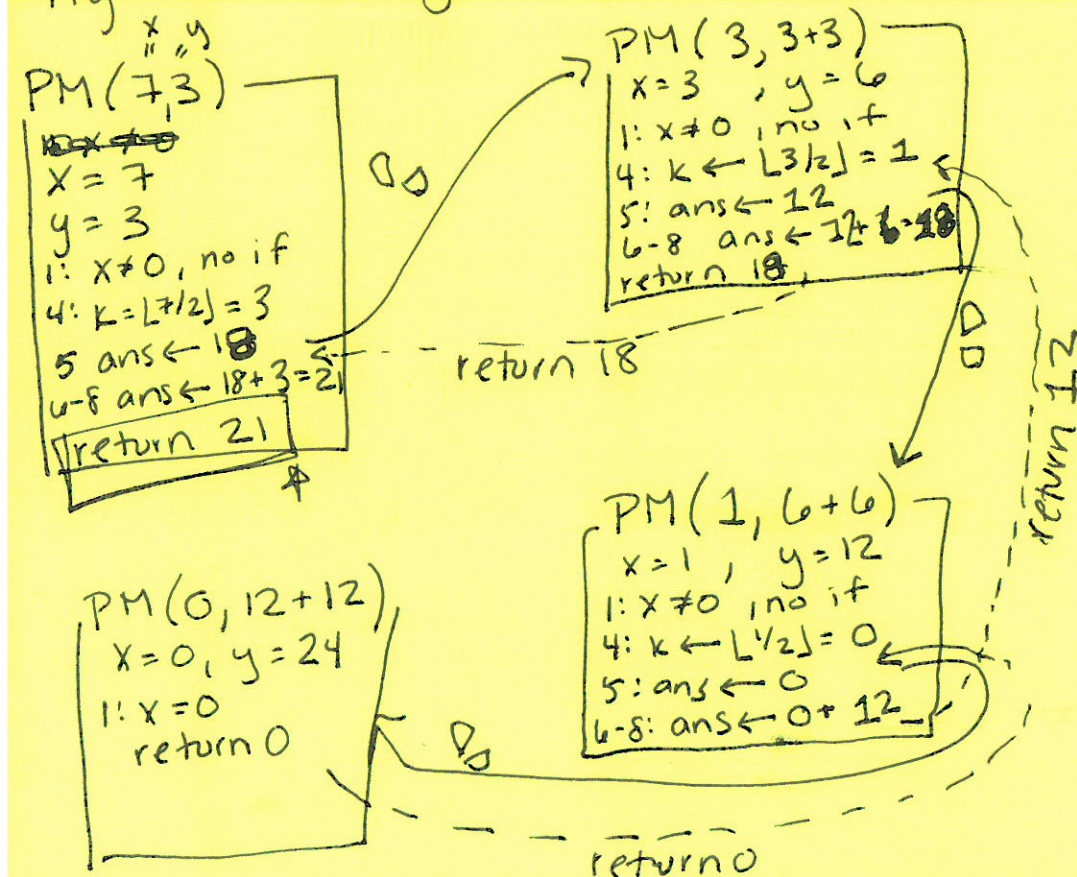
1: if x = 0
2: | return 0
3: endif

4: k ← ⌊x/2⌋
5: ans ← PEASANT MULT(k, y+y)
6: if x is odd
7: | ans += y
8: endif
9: return ans
    
```

x is even
 $\Rightarrow x = 2k$ for some $k \in \mathbb{Z}$
 $(x)(y) = (2k)(y)$
 $= (k)(2y)$
 $= (k)(y+y)$

x is odd
 $\exists k \in \mathbb{Z} \text{ s.t. } x = 2k+1$
 note: $k = \lfloor x/2 \rfloor$
 $(x)(y) = (2k+1)(y)$
 $= (2k)(y) + y$
 $= (k)(y+y) + y$

Try: walk through PEASANT MULTIPLY (7, 3).



NOTES:

$\mathbb{R} \rightarrow \mathbb{Z}$
 Floor: $\lfloor z \rfloor \in \mathbb{Z}$
 rounds down to next integer
 Ceil: $\lceil z \rceil$
 rounds up

$$\lfloor 1.2 \rfloor = 1$$

$$\lceil 1.2 \rceil = 2$$

$$\lfloor \pi \rfloor = 3$$

$$\lceil \pi \rceil = 4$$

$$\lceil 6 \rceil = 6$$

$$\lfloor 6 \rfloor = 6$$

What is the RT of PM? (recursive version)

$T(n)$ = the runtime for PEASANT MULTIPLY $(n, *)$
Shortened to $PM(n, *)$

$$T(n) = \underbrace{\Theta(1)}_{\text{Lines 1-3}} + \underbrace{\Theta(1)}_{\text{Line 4}} + \underbrace{T(\lfloor n/2 \rfloor)}_{\text{Line 5}} + \underbrace{\Theta(1)}_{\text{Lines 6-9}}$$

$$= T(\lfloor n/2 \rfloor) + \Theta(1)$$

recurrence relation: $T(n) = T(\lfloor n/2 \rfloor) + \Theta(1)$

Sometimes written: $T(n) = T(n/2) + \Theta(1)$

asymptotic form: $T(n) \in \Theta(\log n)$

↑
Same recurrence
as binary
search

CSCI 432 Handout 05: Recursion Invariants

Name: CLASS

Collaborators: _____

3 February 2026

Recursion Invariants: Proving Partial Correctness

Recall the HANOI algorithm (given in Algorithm 1); see [1, Ch. 1]. In this problem, have three towers, each with a set of discs on them. The stacking rule is that all discs are a unique size, all disks are stacked on towers, and no smaller disc is underneath a larger one.

Algorithm 1 HANOI($n, src, dest, tmp$)

Input: $n \in \mathbb{N}$, and three towers with disks: $src, dest, tmp$ such that [[See Q1 below]].

Output: [[See Q2 below]].

```
1: if  $n > 0$  then
2:   HANOI( $n - 1, src, tmp, dest$ )
3:   move top disk from  $src$  to  $dest$ 
4:   HANOI( $n - 1, tmp, dest, src$ )
5: end if
```

- 1) Suppose we have n disks total. What are the assumptions on the input to the initial call HANOI($n, src, dest, tmp$)?

Answer:

There are n discs, all are on src . (A0)
tmp has no discs (A1)
discs are in allowed order (A2)
dest has discs (A3)

Going forward, we will call these assumptions P (for the preconditions).

VERY FIRST CALL
Assumption on Input

5. **INITIALIZATION** This is like the base case of induction. Colloquially, we ask “Why is this true for the smallest input?” (And, what are those inputs that would allow us to return without a recursive call?) More formally, we can say: If $n_0 = 0$, then after the call to $\text{HANOI}(n_0, A, B, C)$, the recursion invariant R is satisfied.

Answer:

Note: sometimes, just as in induction, there may be more than one base case.

6. **MAINTENANCE:** This part is JUST like the inductive step of induction. Let $k \in \mathbb{N}$ such that $k \geq n_0$. Assume that, for all $k' \in \mathbb{N}$ such that $n_0 \leq k' \leq k$, the recursion invariant R holds after a call to $\text{HANOI}(k', *, *, *)$. (That was the equivalent to the inductive assumption). Now, we must prove that R holds after $\text{HANOI}(k + 1, A, B, C)$. (Hint: for this, we will almost always need to use the line numbers as we walk through the algorithm to explain how the maintenance step works).

Answer:

7. **END:** This is where we diverge from induction. Because algorithms are finite, we can't go on forever. Colloquially, we say “if the initial call $\text{HANOI}(n, \text{src}, \text{dest}, \text{tmp})$ finishes executing, then all n disks (which were initially on src) have moved to dst .” More formally, we can phrase this as: If the recursion invariant holds, if the preconditions P are satisfied, and if the algorithm completes execution, then the post-condition Q is satisfied. (For shorthand, I might write $R \wedge P \wedge T \implies Q$).

Answer: